Sovereign AI at runtime

Control, governance and performance for AI workloads that matter.



THE CASE IN ONE PAGE

The central argument

A sovereign AI program needs control over the environment where models run, plus efficient use of the compute inside that boundary.

Three connected ideas

Runtime control

Authority must extend to identity, keys, operations, telemetry, model access and inference, alongside data residency.

Hardware choice

A multi-vendor GPU estate can give operators more options, provided orchestration and serving can use those resources well.

Inference efficiency

Cache-aware routing reduces repeated prompt processing and can improve capacity and response time for suitable workloads.

What the evidence establishes

IBM describes Sovereign Core as a software stack for sovereign applications, data and AI. NxtGen publicly describes a GPU platform spanning NVIDIA, AMD and Intel with Red Hat OpenShift. IBM Research reports a joint experiment with Red Hat and NxtGen in which llm-d improved mixed-GPU inference performance. [1-3]

EVIDENCE NOTE

The architecture shown later in this e-book is a discussion model. The cited experiment does not establish a packaged IBM-NxtGen integrated product.

Sources: 1, 2, 3

READER GUIDE

Inside this e-book

The story moves from sovereign control to hardware operations, then to inference performance and a practical pilot.

01	Sovereignty at runtime	04
02	The software foundation	05
03	The compute environment	06
04	Where inference loses capacity	07
05	How llm-d changes serving	08
06	What the experiment found	09
07	Enterprise implications	10
08	A joint discussion model	11
09	A path to production	12
10	Decision worksheet	13
	Sources and definitions	14

EVIDENCE KEY

Product descriptions are attributed to the companies. Research results are tied to the reported experiment. The operating model is an illustrative synthesis for discussion.

Sovereignty at runtime

Data location is only one part of the question. The operator also needs to know who controls the AI system while requests are processed.

For production AI, the relevant boundary reaches across the platform. An organization may keep data in an approved location yet still lack clear authority over administrators, keys, model access, logs or runtime policy. IBM frames sovereignty as governance, control and compliance built into the deployment model. [1]

DATA

Where sensitive data resides and how it moves.

OPERATIONS

Who administers the environment and under which authority.

TECHNOLOGY

Which infrastructure, platform and models can be replaced or extended.

AI RUNTIME

Where inference runs, who may use it and how actions are governed.

EVIDENCE NOTE

The four dimensions synthesize IBM’s published description. They are useful questions for architecture review, not a certification framework.

Sources: 1

02 / SOFTWARE FOUNDATION

IBM Sovereign Core

IBM presents Sovereign Core as an integrated software stack for applications, data and AI under customer control. [1]

SOVEREIGN BOUNDARY

Customer operated control

Control plane, identity and keys

Governed services

AI inference and extensible catalog

Continuous evidence

Policy, monitoring and audit inputs

INFRASTRUCTURE CHOICE

The software foundation is intended to work across infrastructure selected for the sovereign environment. The operating boundary, responsibilities and available services still require deployment-specific design. [1]

EVIDENCE NOTE

This page describes IBM’s published product positioning. It does not assert that Sovereign Core is already integrated with NxtGen’s platform.

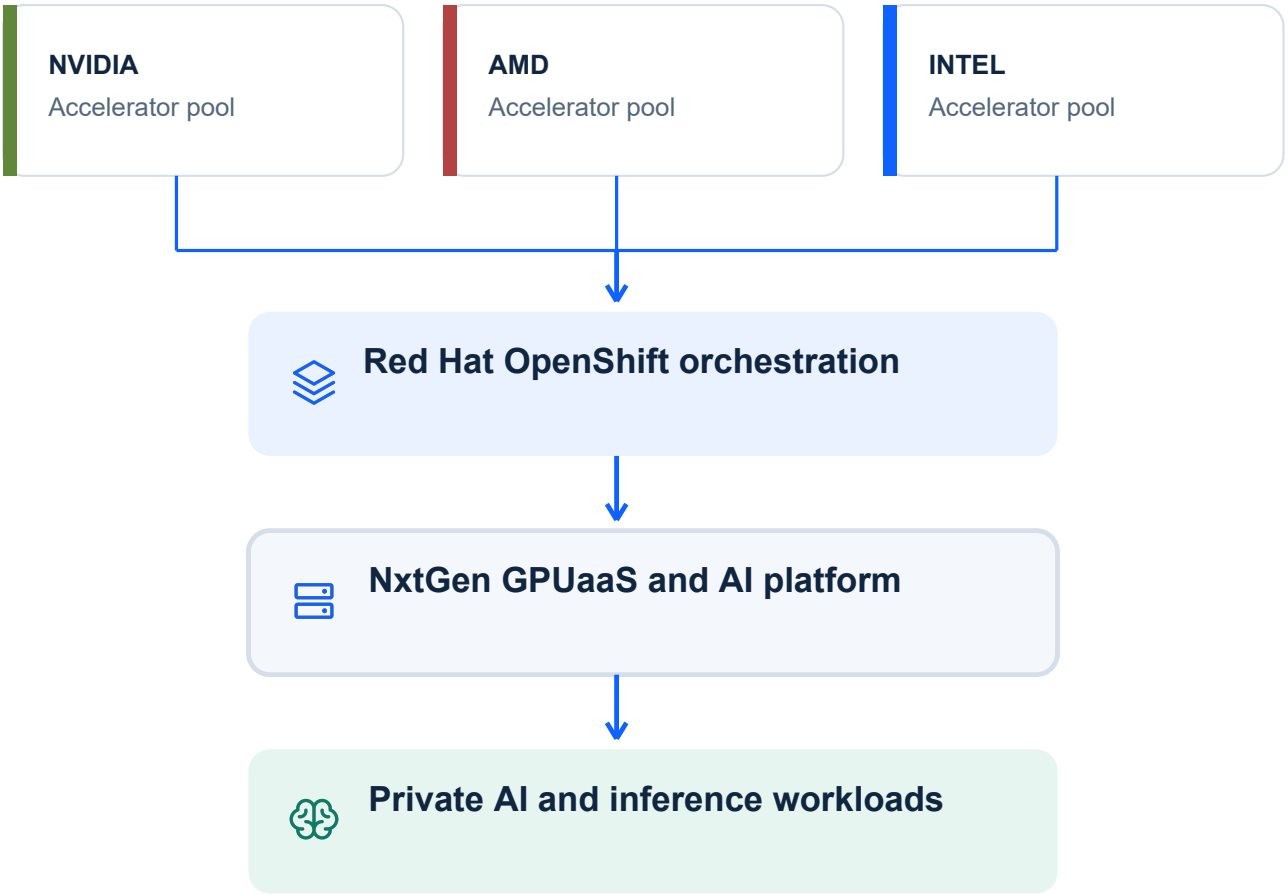
Sources: 1

03 / COMPUTE ENVIRONMENT

A heterogeneous GPU estate

NxtGen positions its GPU-as-a-Service platform around compute choice across vendors, unified with Red Hat OpenShift. [3]

NxtGen's public description names NVIDIA, AMD and Intel accelerators. The operational challenge is to expose that varied pool to AI workloads without forcing each application to handle hardware differences by itself. [3]



EVIDENCE NOTE

This diagram summarizes public platform positioning. The specific services, controls and GPU mix available to a customer depend on the deployment.

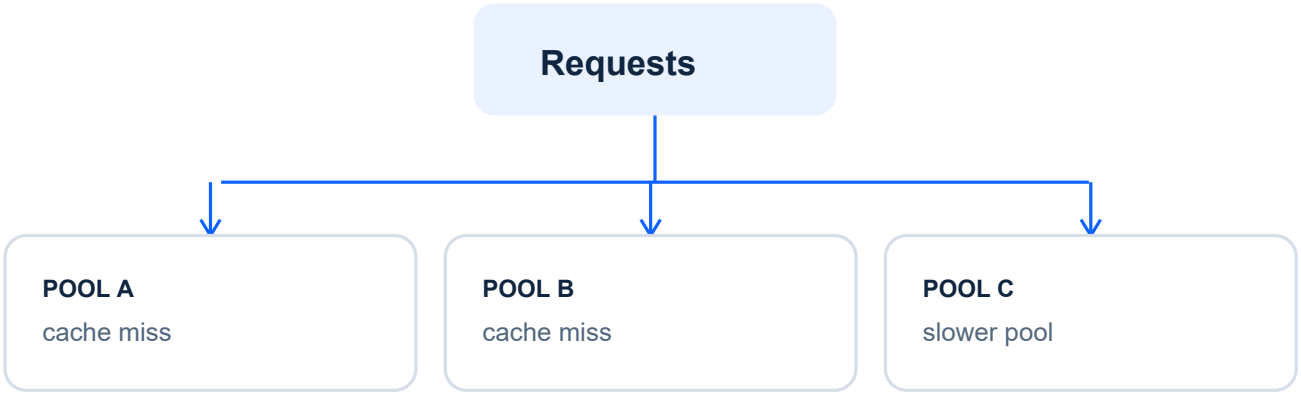
Sources: 3

Where capacity is lost

Round-robin routing can send related prompts to different instances, causing the same context to be computed again. [2]

A language model keeps intermediate attention state in a key-value (KV) cache. When the next request lands on an instance with matching cached context, it can reuse work. When it lands elsewhere, the instance may rebuild that context. Mixed GPU speeds can make even distribution less effective under load. [2]

A SIMPLIFIED ROUND-ROBIN PATH



OPERATIONAL EFFECT

Repeated prompt computation consumes GPU time. At higher traffic, latency can rise while usable capacity falls. The size of the effect depends on the workload and cache behavior.

Why the serving layer matters

The router can use cache state and hardware availability to select a better destination. That turns the same GPU estate into a more capable service without assuming every accelerator has identical performance. [2]

EVIDENCE NOTE

The diagram explains a mechanism. It is not a measured result from a particular customer environment.

Sources: 2

How llm-d changes the path

IBM Research describes llm-d as a Kubernetes-native serving layer with cache-aware routing and separate prefill and decode stages. [2]

1. **Request**
   Prompt and context enter the service.
2. **Route**
   Select an instance likely to hold matching KV cache.
3. **Prefill**
   Process prompt context, a compute-heavy stage.
4. **Decode**
   Generate tokens, a memory-heavy stage.

EVIDENCE NOTE

The cited research demonstrated gains on a mixed-vendor pool. Cross-vendor optimization of the prefill/decode split remains a development direction in the cited article.

Sources: 2

What the experiment found

IBM Research, Red Hat and NxtGen tested llm-d on a 20-pod pool spanning three GPU vendors in the NxtGen sovereign cloud. [2]

OUTPUT AT HEAVY TRAFFIC

Reported tokens per second for the same heterogeneous pod pool



The traditional setup reached about 9,600 tokens/s under moderate traffic, then fell to 7,500 as demand increased. With llm-d, output reached 14,200 tokens/s under heavy traffic. IBM also reported response time falling by nearly half a minute under heavy traffic. [2]

ADDITIONAL REPORTED FINDING

3-5x faster serving for the tested IBM Granite and Sarvam AI models, with potential to serve twice as many users. [2]

EVIDENCE NOTE

Experiment-specific results. Hardware, model, prompts, traffic and service objectives must match before projecting these gains to another workload.

Sources: 2

What this could change

The research suggests a path to better use of existing compute, provided the gains hold for the organization’s own workloads.

Use available accelerators

Mixed generations and vendors may serve different workload priorities when software support allows it.

Reduce avoidable wait time

Cache-aware routing can limit repeated prefill work and improve the user experience under load.

Increase useful capacity

Higher throughput can defer some incremental hardware demand, depending on price and utilization.

Keep authority visible

The infrastructure benefit matters only if access, telemetry, data and models stay within the defined operating boundary.

VALIDATE BEFORE COMMITTING

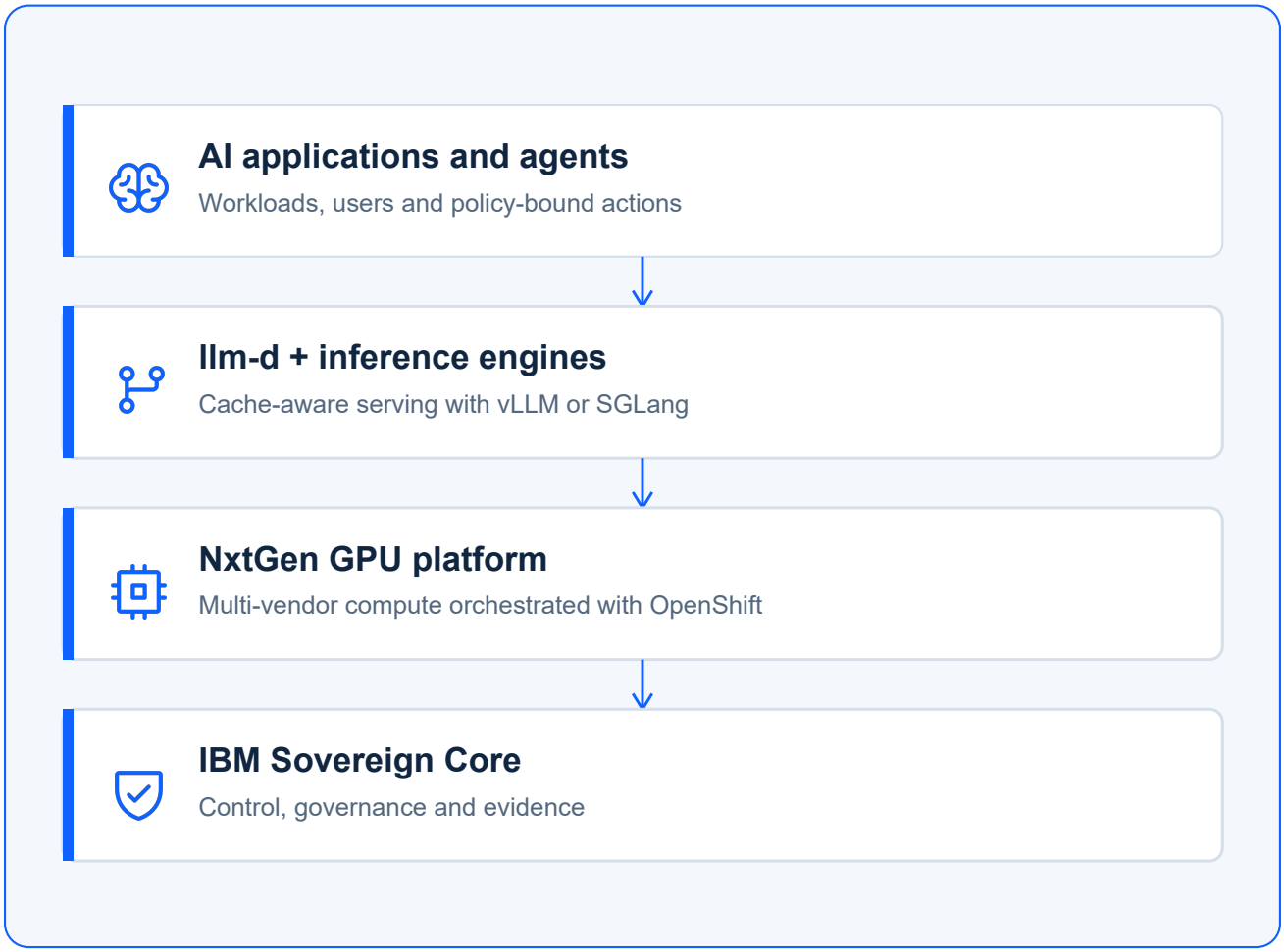
Measure throughput, time to first token, latency distribution, cost per request and governance evidence on representative prompts and the actual GPU mix. Compare against a baseline with the same demand pattern.

Sources: 1, 2, 3

The operating model

An illustrative architecture connects sovereign governance, NxtGen infrastructure and cache-aware inference for discussion.

ILLUSTRATIVE SOVEREIGN BOUNDARY



EVIDENCE NOTE

Illustrative architecture for event discussion. The cited sources document the individual products and the joint llm-d experiment, but do not confirm this as a packaged, supported product integration.

Sources: 1, 2, 3

A focused pilot

A single well-chosen workload can test both the control boundary and the performance case before a broader rollout.

01

Assess

Define the data boundary, model, traffic pattern, target users and service-level objectives.

02

Benchmark

Run representative prompts on the available GPU mix. Record baseline throughput, latency and cost.

03

Pilot

Operate a governed AI service with evidence collection, access control and performance monitoring.

04

Scale

Document operating roles and repeatable deployment patterns after the pilot meets agreed criteria.

SESSION OUTCOME

Name one workload and one measurable outcome. Then assign owners for the boundary, benchmark and pilot decision.

Sources: 1, 2

Questions to settle first

Use these prompts to turn the event conversation into a specific architecture and test plan.

01 WORKLOAD

Which application has a clear business owner, representative prompts and a meaningful response-time target?

02 AUTHORITY

Who administers the environment, identity, keys, models and audit evidence?

03 BOUNDARY

Which data may cross the boundary, and what must remain inside it?

04 COMPUTE

Which accelerators are available, and what is their current utilization and cost?

05 SUCCESS

What measured gain in latency, throughput or cost would justify production use?

Recommended first decision: choose one high-volume inference use case with repeated context. It will make cache reuse and governance easier to measure.

REFERENCE NOTES

Sources and definitions

The main product and research claims are tied to the public sources below.

1 **IBM Sovereign Core**

Product description for governance, control, compliance and AI services.

https://www.ibm.com/products/sovereign-core

2 **IBM Research, “How to run AI workloads on mixed GPUs quickly and affordably”**

Published 23 June 2026. Joint experiment, llm-d mechanism and reported benchmark.

https://research.ibm.com/blog/fast-inference-mixed-gpus

3 **NxtGen, GPU-as-a-Service**

Public positioning for multi-vendor GPUs and Red Hat OpenShift.

https://www.nxtgen.com/gpu-as-a-service

Terms used in this e-book

KV CACHE	Stored intermediate attention state that can reduce repeated prompt processing.
PREFILL	Processing the input prompt and context before output tokens are generated.
DECODE	Generating output tokens after prefill.
SLO	Service-level objective, such as response time or throughput target.

ICON CREDIT AND LICENSE

Phosphor Icons (regular weight). MIT License. Copyright (c) 2023 Phosphor Icons. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.